

std::uninitialized<T>

Document #: P3074R1
Date: 2024-01-30
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 The uninitialized storage problem](#)
- [3 Design Space](#)
 - [3.1 A library_type: std::uninitialized<T>](#)
- [4 Wording](#)
- [5 References](#)

§ 1 Revision History

[[P3074R0](#)] originally proposed the function `std::start_lifetime(p)`. This revision adds a new section discussing the [uninitialized storage problem](#), which motivates a change in design to instead propose `std::uninitialized<T>`.

§ 2 Introduction

Consider the following example:

```
template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(u.storage, u.storage+size);
    }

    constexpr auto push_back(T const& v) -> void {
        std::construct_at(u.storage + size, v);
        ++size;
    }
};
```

```

    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
}
static_assert(silly_test() == 1);

```

This is basically how any static/non-allocating/in-place vector is implemented: we have some storage, that we *definitely do not value initialize* and then we steadily construct elements into it.

The problem is that the above does not work (although there is [implementation divergence](#) - MSVC and EDG accept it and GCC did accept it even up to 13.2, but GCC trunk and Clang reject).

Getting this example to work would allow `std::inplace_vector` ([P0843R9]) to simply work during `constexpr` time for all times (instead of just trivial ones), and was a problem briefly touched on in [P2747R0].

§ 2.1 The uninitialized storage problem

A closely related problem to the above is: how do you do uninitialized storage? The straightforward implementation would be to do:

```

template <class T>
struct BufferStorage {
private:
    alignas(T) unsigned char buffer[sizeof(T)];

public:
    // accessors
};

```

This approach generally works, but it has two limitations:

1. it cannot work in `constexpr` and that's likely a fundamental limitation that will never change, and
2. it does not quite handle overlapping objects correctly.

What I mean by the second one is basically given this structure:

```

struct Empty { };

struct Sub : Empty {
    BufferStorage<Empty> buffer_storage;
};

```

If we initialize the `Empty` that `buffer_storage` is intended to have, then `Sub` has two subobjects of type `Empty`. But the compiler doesn't really... know that, and doesn't adjust them accordingly. As a result, the `Empty` base class subobject and the `Empty` initialized in `buffer_storage` are at the same address, which violates the rule that all objects of one type are at unique addresses.

An alternative approach to storage is to use a `union`:

```
template <class T>
struct UnionStorage {
private:
    union { T value; };

public:
    // accessors
};

struct Sub : Empty {
    UnionStorage<Empty> union_storage;
};
```

Here, now the compiler knows for sure there is an `Empty` in `union_storage` and will lay out the types appropriately. See also [gcc bug 112591](#).

So it seems that the `UnionStorage` approach is strictly superior: it will work in `constexpr` and it lays out overlapping types properly. But it has limitations of its own. As with the `FixedVector` example earlier, you cannot just start the lifetime of `value`. But also in this case we run into the `union` rules for special member functions: a special member of a `union`, by default, is either trivial (if that special member for all alternatives is trivial) or deleted (otherwise). Which means that `UnionStorage<std::string>` has both its constructor and destructor *deleted*.

We can work around this by simply adding an empty constructor and destructor (as shown earlier as well):

```
template <class T>
struct UnionStorage2 {
private:
    union U { U() { } ~U() { } T value; };
    U u;

public:
    // accessors
};
```

This is a fundamentally weird concept since `U` there has a destructor that does nothing (and given that this is a class to be used for uninitialized storage), it *should* do nothing - that's correct. But that destructor still isn't trivial. And it turns out there is still a difference between “destructor that does nothing” and “trivial destructor”:

Trivially Destructible	Non-trivially Destructible
<pre>struct A { }; auto alloc_a(int n) -> A* { return new A[n]; } auto del(A* p) -> void { delete [] p; }</pre>	<pre>struct B { ~B() { } }; auto alloc_b(int n) -> B* { return new B[n]; } auto del(B* p) -> void { delete [] p; }</pre>

Trivially Destructible	Non-trivially Destructible
<pre> alloc_a(int): movsx rdi, edi jmp operator new[] (unsigned long) del(A*): test rdi, rdi je .L3 jmp operator delete[] (void*) .L3: ret </pre>	<pre> alloc_b(int): movabs rax, 9223372036854775800 push rbx movsx rbx, edi cmp rax, rbx lea rdi, [rbx+8] mov rax, -1 cmovb rdi, rax call operator new[](unsigned long) mov QWORD PTR [rax], rbx add rax, 8 pop rbx ret del(B*): test rdi, rdi je .L9 mov rax, QWORD PTR [rdi-8] sub rdi, 8 lea rsi, [rax+8] jmp operator delete[](void*, unsigned long) .L9: ret </pre>

That's a big difference in code-gen, due to the need to put a cookie in the allocation so that the corresponding `delete[]` knows how many elements there so that their destructors (even though they do nothing!) can be invoked.

While the union storage solution solves some language problems for us, the buffer storage solution can lead to more efficient code - because `StorageBuffer<T>` is always trivially destructible. It would be nice if he had a good solution to all of these problems - and that solution was also the most efficient one.

§ 3 Design Space

A previous revision of this paper [[P3074R0](#)] talked about three potential solutions to this problem:

1. a library solution (add a `std::uninitialized<T>`)
2. just make it work (change the union rules to implicitly start the lifetime of the first alternative, if it's an implicit-lifetime type)
3. add a library function to explicitly start lifetime

That paper proposed a new function `std::start_lifetime(p)` that was that third option. However, with the addition of the overlapping subobjects problem and the realization that the union solution has

overhead compared to the buffer storage solution, it would be more desirable to solve both problems in one go.

Now, (2) is no longer a meaningful option because we can't really "just make it work" since attempting to change the union rules with regards to trivial destruction would be quite a language change and an ABI break.

Similarly, (3) doesn't really address the storage problem. In order to make the `union` solution work out, we'd need to add another way to tell the compiler that we want our constructor and destructor to be trivial - not deleted - regardless of the alternatives. Perhaps that ends up being something like this:

```
template <class T>
struct UnionStorageFuture {
    union { [[uninitialized]] T value; };
    constexpr UnionStorageFuture() { std::start_lifetime(&value); }
};
```

However, now we need two features - some way to mark the alternative and some way to start its lifetime. That, inherently, doesn't seem like a great solution.

Which seems to leave only one solution to the problem, which is easier to use anyway:

§ 3.1 A library type: `std::uninitialized<T>`

We could introduce another magic library type, `std::uninitialized<T>`, with an interface like:

```
template <typename T>
struct uninitialized {
    union { T value; };
};
```

As basically a better version of `std::aligned_storage`. Here is storage for a `T`, that implicitly begins its lifetime if `T` is an implicit-lifetime-type, but otherwise will not actually initialize it for you - you have to do that yourself. Likewise it will not destroy it for you, you have to do that yourself too. This type would be specified to always be trivially default constructible and trivially destructible. It would be trivially copyable if `T` is trivially copyable, otherwise not copyable.

`std::inplace_vector<T, N>` would then have a `std::uninitialized<T[N]>` and go ahead and `std::construct_at` (or, with [P2747R1](#), simply placement-new) into the appropriate elements of that array and everything would just work.

Because the language would recognize this type, this would also solve the overlapping objects problem.

§ 4 Wording

Add to 20.2.2 [\[memory.syn\]](#):

```
namespace std {
    // ...
```

```

// [obj.lifetime], explicit lifetime management
template<class T>
    T* start_lifetime_as(void* p) noexcept;
    // freestanding
template<class T>
    const T* start_lifetime_as(const void* p) noexcept;
    // freestanding
template<class T>
    volatile T* start_lifetime_as(volatile void* p) noexcept;
    // freestanding
template<class T>
    const volatile T* start_lifetime_as(const volatile void* p) noexcept;
    // freestanding
template<class T>
    T* start_lifetime_as_array(void* p, size_t n) noexcept;
    // freestanding
template<class T>
    const T* start_lifetime_as_array(const void* p, size_t n) noexcept;
    // freestanding
template<class T>
    volatile T* start_lifetime_as_array(volatile void* p, size_t n)
        noexcept; // freestanding
template<class T>
    const volatile T* start_lifetime_as_array(const volatile void* p,
        // freestanding
        size_t n) noexcept;

+ template<class T>
+ struct uninitialized;
+     // freestanding
}

```

With corresponding wording in 20.2.6 [\[obj.lifetime\]](#):

- 9 The class `uninitialized` is suitable for storage for an object of type `T` that is not initially initialized.

```

template<class T>
struct uninitialized {
    union { T value };

    constexpr uninitialized();
    constexpr uninitialized(const uninitialized&);
    constexpr uninitialized& operator=(const uninitialized&);
    constexpr ~uninitialized();
};

```

- 10 `uninitialized<T>` is a trivially default constructible and trivially destructible type.

- 11 [*Note 1*: An object of type `T` and the value subobject of `uninitialized<T>` have distinct addresses ([intro.object]) — *end note*]

```

constexpr uninitialized();

```

12 *Effects:* If τ is an implicit-lifetime type, begins the lifetime of value. Otherwise, none. [*Note 2:* The constructor of τ , if any, is not called — *end note*]

```
constexpr uninitialized(const uninitialized&);  
constexpr uninitialized& operator=(const uninitialized&);
```

13 If τ is a trivially copyable type, then `uninitialized<T>` is a trivially copyable type. Otherwise, `uninitialized<T>` is not copyable.

```
constexpr ~uninitialized();
```

14 *Effects:* None. [*Note 3:* The destructor of τ , if any, is not called — *end note*]

§ 5 References

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. `inplace_vector`.

<https://wg21.link/p0843r9>

[P2747R0] Barry Revzin. 2022-12-17. Limited support for `constexpr void*`.

<https://wg21.link/p2747r0>

[P2747R1] Barry Revzin. 2023. ``constexpr` placement new`.

<https://wg21.link/p2747r1>

[P3074R0] Barry Revzin. 2023-12-15. `constexpr union lifetime`.

<https://wg21.link/p3074r0>